

赤外線マスク同期を用いた 複数 Arduino による輪唱演奏システム

情報工学科 2 年

25G1046

櫛濱 和輝

—— チーム 9 ——

手代木 巧 (24G1098) 久家 力孔 (24G1052)

渡邊 乃斗 (24G1145) 家田 祐希 (25G1010)

木下 遼介 (25G1043)

2026 年 7 月 9 日

1 はじめに

近年、IoT 技術の普及によって、複数のデバイスがネットワークを介してリアルタイムに連携するシステムが様々な分野で実用化されている。製造業における機械の協調制御やスマートホームの機器連携など、複数の自律した機器を同期させる技術は現代の組み込みシステムに広く求められている。合奏はこの同期という課題が音として直接聞こえる題材である。Rasch によると、合奏において奏者間の発音タイミングのずれはおよそ 30~50 ms の範囲に収まっており、これを大きく超えるずれは聴き手に同期の乱れとして知覚される [1]。つまり、複数の機器で合奏を成立させるには、数十 ms 以内の精度で状態を共有し続ける必要がある。

電子楽器の同期に関する既存技術としては、MIDI や Wi-Fi・Bluetooth を用いたシステムが挙げられる。MIDI は音高やタイミング、強さといった精密な情報を伝達できる規格である [2] が、専用のインターフェースと対応ライブラリが必要であり、同期のきっかけだけを伝えたい本プロジェクトの用途には過剰な仕様となる。Wi-Fi や Bluetooth を用いた合奏システムは遅延補正の技術も発達しているものの、通信インフラの品質に動作が左右されるため、多数の機器が電波を発する実験室環境では干渉や接続不安定のリスクが残る。そこで本プロジェクトでは、伝達する情報を「どの楽器がいま演奏に参加しているか」という最小限の同期情報に限定し、赤外線通信で実現する方針を採った。赤外線はネットワーク環境を必要とせず、LED と受光モジュールだけで構成できるため、小規模な組み込みシステムに低コストで導入できる [3]。

本プロジェクトでは、5 台の Arduino を用いたオーケストラシステム「赤外線通信で歌うカエル」を制作した。1 台を指揮側の親機、4 台をピアノ・フルート・トランペット・ドラムを担当する演奏側の子機とし、楽曲「かえるのうた」を輪唱形式で演奏する。親機は演奏中の楽器集合を表すマスクデータを赤外線で全子機へ一斉送信し、各子機は自分の担当ビットが有効なときにのみ譜面を 1 コマ進め、PC 上の Processing へ音符情報を送って楽器音を再生させる。演奏者はボタン操作で各楽器を任意のタイミングで参加・退場させることができ、可変抵抗によるテンポの連続調整にも対応する。さらに子機に接続した LCD には、かえるの口パクアニメーションと歌詞を表示する。

報告者は本システムのうち、ピアノ音源の合成と Processing 側の再生処理、ピアノ担当子機と Arduino との通信用関数、ドラム担当子機のプログラム、ブレッドボードの設計、およびテンポ管理プログラムの実装を担当した。テンポ管理は渡邊との共同担当である。また、開発終盤には親機と子機で楽譜 1 周分の tick 数が食い違い特定の音が周期的に抜けるバグの修正と、休符が休符として聞こえない問題の修正を行った。

評価では、単体テストで各楽器音色と通信機能を確認したのち、第 9 週末に親機 1 台と子機 4 台による輪唱の実機テストを実施し、ボタン押下順に楽器が参加するカノン演奏が音抜けや休符抜けなく成立することを確認した。演奏タイミング誤差と赤外線通信成功率の定量計測は、計測用の同期パルス出力とログ機構を実装済みである。本報告書では、システム全体の構成と報告者の担当部分の実装を述べたうえで、実機テストの結果と残された課題について考察する。

2 作成したシステムの概要

本システムは、同期制御機能、楽曲制御機能、および LCD 表示機能から構成される。同期制御は親機からの赤外線信号の送受信、楽曲制御は音符情報の生成・演奏タイミング制御・音声出力、LCD 表示はかえるのアニメーションと歌詞のスクロール表示をそれぞれ担う。まずシステム全体の構成を説明する。

2.1 システム構成

システムの構成を図 1 に示す。本システムは、指揮側の親機 Arduino 1 台、演奏側の子機 Arduino 4 台、および楽器音を再生する PC 4 台から構成される。いずれの Arduino も Arduino Uno R4 を用いた。親機には赤外線 LED、演奏開始・リセット・子機参加用のタクトスイッチ、およびテンポ調整用の可変抵抗を接続する。各子機には赤外線受光モジュール OSRB38C9AA[7] を接続し、USB シリアルで PC と接続する。PC 側では Processing が動作し、子機から届いた音符情報をもとに楽器音を合成してスピーカから出力する。楽器音の生成には Processing のサウンドライブラリである Minim[5] を用いた。

システム全体のデータの流れは一方方向である。親機は現在の tick 番号と参加楽器マスクを赤外線でブロードキャストし、子機はそれを受けて自分の譜面配列を参照し、該当する音符情報を Processing へ送信する。Processing は受信した音符情報から楽器音を合成して再生する。子機は LCD への表示出力も並行して行う。

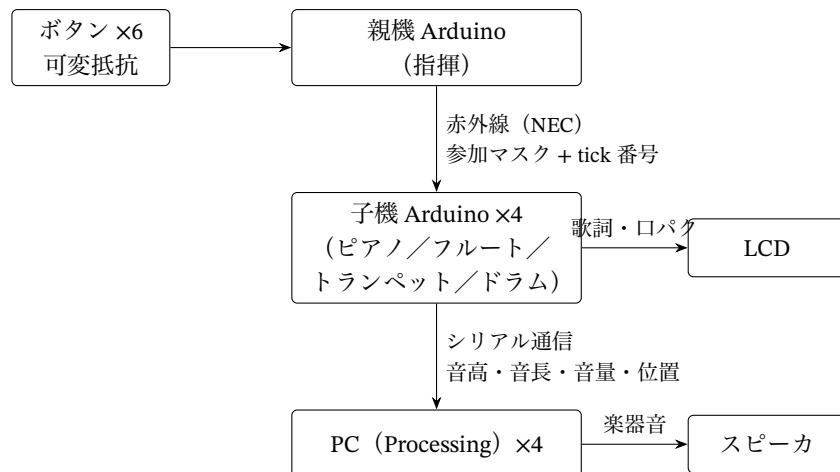


図 1 システム構成とデータの流れ

2.2 赤外線同期プロトコル

親機と子機の間同期には、家電リモコンで広く使われる NEC フォーマット [3] を用いた。送受信には Arduino 用ライブラリの IRremote[4] を利用している。NEC フォーマットは 8 ビットのア

ドレスと 8 ビットのコマンドを 1 フレームで送る規格であり，本システムではコマンド部に演奏中の楽器集合を表すマスクを，アドレス部に親機の tick カウンタの下位 8 ビットを載せる．マスクはビット i が子機 i の参加状態に対応し，各子機は受信したマスクのうち自分の担当ビットだけを参照して演奏可否を判定する．楽譜のデータそのものは送らず，各子機が自分のパートの譜面配列を保持する分散構成とした．これにより 1 tick あたりの通信は 1 フレームで済み，子機の台数が増えても通信量が変わらない．

親機は演奏中，テンポに応じた周期で tick を刻む．tick 間隔 T_{tick} はテンポを BPM として

$$T_{\text{tick}} = \frac{60000}{2BPM} \text{ [ms]} \quad (1)$$

であり，1 tick が 8 分音符 1 個に相当する．120 BPM のとき T_{tick} は 250 ms になる．親機は参加中の子機が 1 台以上ある tick でのみ赤外線フレームを送信する．

2.3 輪唱の実現と参加・退場の状態管理

輪唱は，同じ旋律を時間差で重ねることで成立する．本システムでは，親機の再生開始後に演奏者が子機ごとのボタンを押すことで，その楽器を任意のタイミングで演奏に参加させる方式を採用した．人間の入力タイミングにはばらつきがあるため，参加の反映は次の 8 tick 境界，すなわち拍のまとまりの先頭まで待ってから行う．こうすることで，どのタイミングでボタンを押しても各声部の入りが拍に揃い，輪唱としてかみ合う．退場の反映は，その子機が参加した時点を起点として楽譜 1 周分である 32 tick の倍数の境界に揃える．旋律を途中で切らず，1 周弾き切ってから抜けるための設計である．

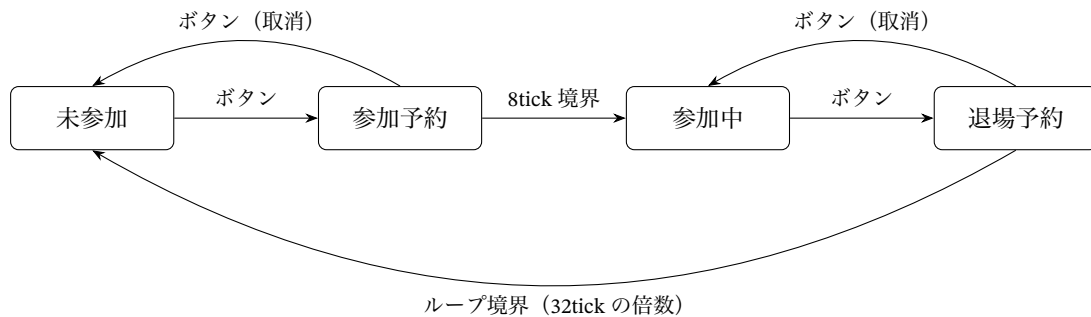


図2 子機 1 台分の参加状態の遷移

親機が管理する子機 1 台分の状態遷移を図 2 に示す．各子機は未参加，参加予約，参加中，退場予約の 4 状態を巡回し，ボタンを押すたびに予約とその取消が交互に切り替わる．予約中にもう一度押せば取り消せるため，押し間違いにも対処できる．親機の各子機ボタンの隣には状態表示用の LED を設け，参加の意思がある状態で点灯するようにした．どの楽器が参加予約中あるいは参加中かを物理的に確認できるので，演奏デモの操作が分かりやすくなる．

2.4 楽器音の合成

Processing 側では、子機から「音高、音長、音量、譜面位置」のカンマ区切りテキストを受信し、楽器ごとの合成処理で音を再生する。ピアノは倍音の加算合成による減衰音、フルートは倍音構造と ADSR エンベロープの調整による持続音、トランペットは倍音比 [1.0, 0.6, 0.35, 0.2, 0.1, 0.05] と 5 Hz のビブラートによる金管らしい音、ドラムはキックとシンバルの 2 種類の打撃音である。ピアノ音源の詳細は報告者の担当分として 3.3 節で述べる。

2.5 LCD 表示

エンターテインメント性の要素として、子機に接続した LCD キャラクターディスプレイに、かえるのドット絵アニメーションと演奏に合わせた歌詞のスクロールを表示する。演奏の進行位置は子機が保持しているため、発音と同じタイミングで表示を更新できる。本報告の執筆時点では歌詞が縦に流れてしまう不具合が残っており、表示部分の統合は最終週に持ち越している。

3 システム実装

3.1 チーム構成と役割分担

本プロジェクトは 6 名で実施した。作業は Processing での音源作成、子機 Arduino のプログラム、親機 Arduino のプログラム、回路・表示装置の 4 系統に分け、WBS の管理番号を付けて分担した。役割分担を表 1 に示す。報告者はピアノ音源の作成とその Arduino 通信用関数、ドラムの子機プログラム、およびテンポ管理プログラムを担当した。テンポ管理は渡邊との共同である。第 9 週末までのチーム全体の作業時間は 129 時間であり、報告者の累計は 35 時間であった。

表 1 チームの役割分担

担当者	WBS 番号	主な担当
櫛濱 和輝 (報告者)	110, 120, 240, 320	ピアノ音源・通信用関数、ドラム子機、テンポ管理 (共同)
家田 祐希	130, 140, 220, 310	フルート音源・通信用関数・子機、赤外線管理 (共同)
木下 遼介	150, 160, 230, 410	トランペット音源・通信用関数・子機、回路 (共同)
久家 力孔	170, 171, 180	ドラム音源 (キック・シンバル)、ドラム通信用関数
渡邊 乃斗	250, 310, 320	楽譜進行、赤外線管理 (共同)、テンポ管理 (共同)
手代木 巧	410, 420	回路作成、LED マトリクス・LCD 表示
全員	510, 520	輪唱機能の結合、結合テスト

開発は、第 5 週までを計画段階、第 6 週から第 9 週までを実装段階とし、第 10 週に統合と結合テストを行う計画で進めた。WBS 第 3 層のタスク単位のスケジュールを図 3 に示す。タスク番号は表 1 の WBS 番号に対応する。

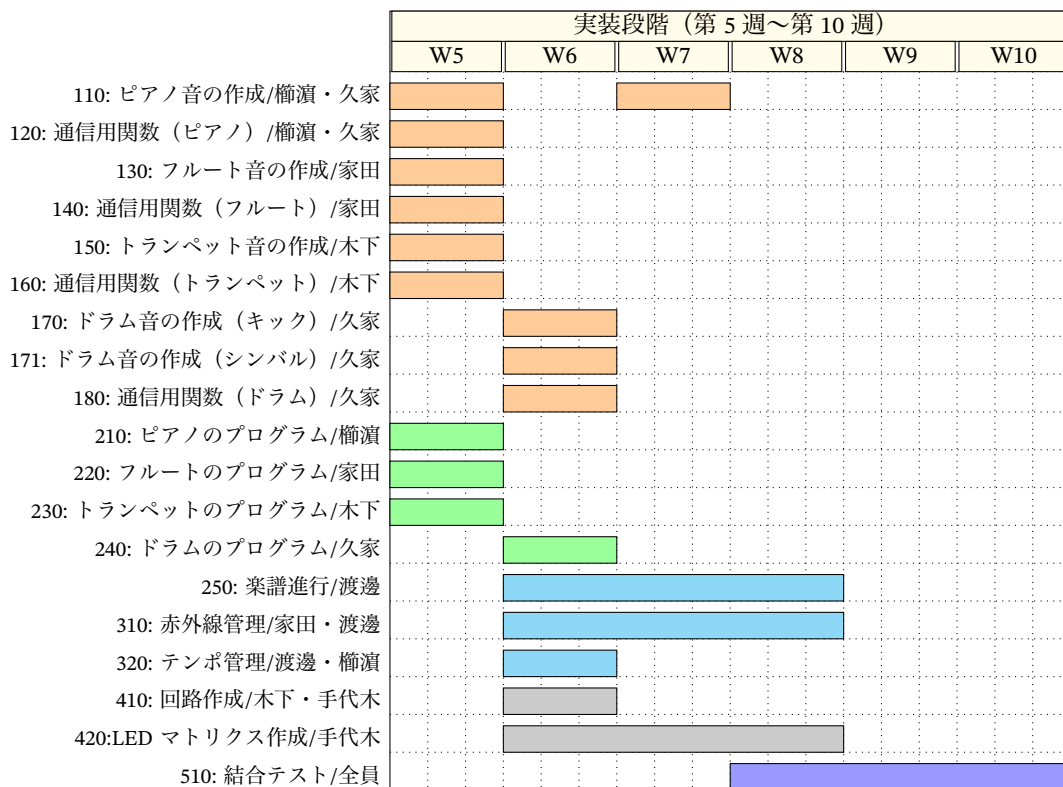


図3 開発スケジュール

図3のバーは計画時の週割当てである。110: ピアノ音の作成は第5週の仮実装と第7週の音色改善の2期間に分けて実施した。実施ではいくつかのタスクが計画より後ろへ延びており、420: LED マトリクス作成は表示不具合の解消に時間を要して第10週まで作業が続いた。250: 楽譜進行、310: 赤外線管理、320: テンポ管理の3タスクも、第8週の結合テストで発覚した課題に対応するため第9週まで追加実装を行った。音源作成と通信用関数は計画通り第7週末までに完了している。

3.2 報告者の担当範囲と技術的課題

報告者の担当のうち、ピアノはこのシステムの旋律を担う中心の楽器である。また、テンポ管理と演奏進行の時間設計はすべての楽器に影響する。実装を進めるなかで、解決すべき課題は次の4点に整理された。

- ・打鍵後に減衰していくピアノらしい音色を、音源データを使わずに合成で作ること
- ・発音時の処理遅延を抑え、演奏のタイミングを崩さないこと
- ・親機と子機の進行が食い違い、特定の音が周期的に抜けるバグを解消すること
- ・休符が休符として聞こえるように、音の余韻を制御すること

以降の節で、それぞれの課題への取り組みを順に述べる。なお、開発支援ツールとして生成AIを利用した。ピアノ音色の合成アルゴリズムの検討や、バグ修正時の原因候補の洗い出しに用いた

が、生成されたコードはそのまま採用せず、実機での聴感確認とシリアルログによる動作確認で検証したうえで、音色を決めるパラメータ類は試聴を繰り返して手動で調整した。検証の具体的な内容は該当する節と4章で述べる。このほか、Arduino 公式ドキュメント [6]、IRremote ライブラリのドキュメント [4]、受光モジュールのデータシート [7] を情報源として参照した。

3.3 ピアノ音源の実装

ピアノの音は、多数の倍音が異なる速さで減衰していく音である。正弦波1本やオルガン風の持続音では鍵盤楽器に聞こえないため、物理的な性質を模した加算合成を実装した。ピアノ弦の倍音周波数は弦の剛性により整数倍からわずかに高い方へずれることが知られており [8]、この性質はインハーモニシティと呼ばれる [9]。\$h\$ 次倍音の周波数 \$f_h\$ を、基本周波数 \$f_s\$ とインハーモニシティ係数 \$B\$ を用いて

$$f_h = hf_s \sqrt{1 + Bh^2} \quad (2)$$

で計算する。\$B\$ は音域に応じて変え、中央ハ付近で \$2 \times 10^{-4}\$ 程度とした。倍音は10次まで重ね、\$h\$ 次倍音の振幅は \$1/h^p\$ に比例させて高次ほど弱くする。指数 \$p\$ は低音域で0.9、高音域で1.4程度になるように基本周波数から連続的に決めた。

減衰のエンベロープは、打鍵直後の速い減衰とその後の緩やかな減衰の2段階を重ねた形とし、時刻 \$t\$ での振幅係数を

$$e(t) = 0.3e^{-t/0.12} + 0.7e^{-t/\tau_h}, \quad \tau_h = \frac{\tau_1}{1 + 0.6(h-1)} \quad (3)$$

とした。\$\tau_1\$ は基本音の減衰時定数であり、残響の長さが低音ほど長くなるように音域から決める。式(3)の \$\tau_h\$ は次数とともに小さくなるので、高次倍音ほど速く消える。これは実際のピアノで打鍵直後は明るく、時間とともに丸い音色に変わっていく振る舞いに対応する。このほか、実機のピアノが1音あたり複数の弦を持つことを模して、音域に応じて1~3本の弦をセント単位でわずかにずらして重ねた。さらに打鍵の瞬間のハンマーノイズとして、最初の15msだけローパスフィルタを通した雑音を加え、立ち上がり3msのアタックと、音長経過後に時定数0.15sで減衰するダンパーリリースを付けている。

もう1つの課題は発音遅延である。上記の合成をすべて発音のたびに計算すると、波形生成に時間がかかり演奏に間に合わないおそれがある。そこで、楽譜に登場する6種類の音高と2種類の音長の組をProcessingの起動時にすべて波形として生成し、MinimのSamplerにキャッシュしておく方式にした。演奏中はキャッシュ済みSamplerのtrigger()を呼ぶだけなので、発音時の処理は音源データの再生と同じで済む。Samplerは同時6声まで発音できる設定とし、前の音の余韻と次の音が自然に重なるようにした。未登録の音高が届いた場合はその場で生成してキャッシュに加える安全網も設けている。なお、このピアノ音源は当初オルガン風の持続音で仮実装しており、第9週にここで述べた合成方式へ差し替えた。差し替え時には楽譜で使う全音高について試聴による検証を行い、全ケースで楽器音として問題ないことを確認している。波形生成部のコードは付録Cに示す。

3.4 演奏進行の不具合修正

結合テストの過程で、演奏の時間管理に関わる不具合を2件修正した。いずれも症状の観察からログで原因を絞り込んだもので、修正自体は小さいが、輪唱の成立には欠かせない修正だった。

1件目は、特定の位置の音が周期的に鳴らない症状である。旋律の途中、譜面位置16や24の音が数周に一度抜けて聞こえた。子機のシリアルログで譜面位置の進み方を追ったところ、親機は32tickを楽譜1周として進行しているのに対し、子機は1周を33tickと解釈していることが分かった。子機側でループ末尾の休みを表す定数を2tickとしていたため、旋律31tickとの合計が33tickになっていたのが原因である。周回のたびに子機の位置が親機に対して1tickずつ後ろへずれ、ずれの蓄積がちょうど一周する周期で特定位置の音が落ちる、という症状につながっていた。この定数を1に修正して両者を32tickに統一し、症状は解消した。第9週の4子機実機テストでも再発していない。

2件目は、休符が休符として聞こえない問題である。本システムの音長の既定値は0.40sであり、120BPMでのtick間隔0.25sより長い。前の音の余韻が次のtickまで残るため、譜面上の休符Rのタイミングでも直前の音が鳴り続け、旋律の切れ目が埋まってしまっていた。そこでProcessing側で休符Rを受信した時点で、発音中の音を即時に停止する処理を入れた。オルガン風音源の時点ではnoteOff()の即時呼び出しとして実装し、ピアノ音源への差し替え後もSamplerのstop()で同じ動作を維持している。音が鳴っている最中に切る処理なので不自然に聞こえる心配があったが、ピアノのダンパーで音を止めた場合と似た切れ方になり、聴感上の違和感はなかった。

3.5 テンポ管理

テンポ管理は、親機のtick周期を演奏中に変更する機能である。当初はボタンの押下でテンポを段階的に増減する方式を実装していたが、デモで聴かせる際にテンポが跳びとびに変わるのは不自然だったため、第9週に可変抵抗による連続調整へ変更した。10kΩのシングルターン可変抵抗をアナログ入力に接続し、読み値 v (0~1023)をテンポの範囲60~150BPMへ線形に対応付ける。

$$BPM(v) = 60 + \left\lfloor \frac{(150 - 60)v}{1023} \right\rfloor \quad (4)$$

読み取りは50ms間隔とし、読み値のノイズによるテンポの揺れを抑えるため、

$$s_n = \frac{3s_{n-1} + v_n}{4} \quad (5)$$

の指数移動平均で平滑化してから式(4)に与える。親機のtick周期の計算は毎ループ現在のテンポ変数を参照しているため、つまみの操作は次のtickから即座に反映される。シリアルモニタへのログ出力は2BPM以上変化したときに限定し、微小な揺らぎでログが埋まらないようにした。

なお、親機の進行処理はdelay()で待つのではなく、millis()で前回tickからの経過時間を監視する方式である。待ち時間の間もボタン入力と可変抵抗の読み取りを続ける必要があるため、テンポ変更・参加操作・tick送信が互いをブロックしない構造になっている。

3.6 輪唱開始方式の試作と設計変更

輪唱の開始方法については、開発の途中で設計を大きく変えた経緯がある。当初の計画では、演奏開始前にボタンで各楽器の順番を登録し、登録順に固定の時間差で追いかける方式を想定していた。報告者はこの系統の親機プログラムに対して、押下順に応じて各子機の出だしのずれを動的に割り当てる実装を試作した。子機固有の固定遅延値として持っていた配列を、押された順番 N に対する割当 $\{0, 8, 16, 20\}$ tick へ読み替えることで、どのボタンを最初に押してもその楽器が先頭声部として位置 0 から歌い出す、本来のカノンの形を実現するものである。

一方で、チーム内の検討により、輪唱の定義は「固定順で参加する」ものではなく「任意の順・任意のタイミングで参加できる」べきだという結論になり、第 7 週末にアーキテクチャを再生開始後の随時参加ボタン方式へ変更した。この方式では子機は演奏していない間も常に曲位置を共有しているため、途中から参加しても、その時点の位置から自然に演奏へ加われる。押下順カノン方式の試作は採用に至らなかったが、2 案を実機で比較したことで、随時参加方式の方が演奏デモとしての自由度が高いことを確認したうえで移行できた。最終版の親機プログラムはこの随時参加方式であり、報告者が試作で得た 8tick 境界への参加アライメントの考え方は、そのまま参加予約の量子化として引き継がれている。

3.7 担当部分以外の実装と工夫

システムの残りの部分について、チームの実装を概観する。赤外線管理プログラムは渡邊と家田が担当し、親機からマスクと tick 番号を一齐送信する方式や、参加・退場を OFF・QUEUED・ACTIVE の 3 状態で管理する仕組みを実装した。楽譜進行プログラムは渡邊が担当し、スタート信号を受けた tick の記録と、現在の tick との比較で出力音を決める機能を追加した。

フルートは家田の担当で、倍音構造と ADSR の調整により息の混じった柔らかい音色を作り、アタックタイムを短くしてテンポへの追従性を改善した。トランペットは木下の担当で、倍音比とビブラートによる音色合成に加え、音の切り替え時に発生していたプチという雑音を、Minim の出力ラインに 0.03s のフェードアウトを入れることで解消した。また、トランペット系では Arduino から 29 音の楽曲データを一括送信するため、シリアル通信速度を 921600 bps として受信配列への格納を確認している。ドラム音源は久家の担当で、キックとシンバルの 2 種を合成し、Arduino から受信した整数値で再生し分ける Processing を実装した。回路と表示装置は手代木の担当で、親機・子機の回路作成と、LCD 表示プログラムの作成を進めた。

回路面では、第 6 週に赤外線受光モジュールの故障が 3 台見つかり、新品への交換と動作確認を家田が行った。こうした部品不良の切り分けができたのは、受光モジュール単体の受信テストを単体テストとして分離していたためである。

4 評価方法と結果

4.1 評価方法

本システムの評価は、計画段階で定義した有効性指標 MOE と性能指標 MOP に基づいて行う。MOE は「違和感なく聞ける」「可変テンポに対応している」「楽器音らしい音色である」「歌詞・アニメーションが演奏に対応している」の 4 項目であり、それぞれに対応する MOP を表 2 のように定めた。演奏タイミング誤差の目標値 30 ms は、合奏で聴衆がずれを知覚し始める閾値が 20～30 ms 程度とされていること [1] を根拠とする。LCD 表示の同期許容範囲 100 ms は、楽器演奏における音と映像のずれの知覚に関する報告 [10] に基づく。

表 2 性能指標 MOP と目標値

MOP	性能指標	目標値	第 9 週末時点の状態
MOP1	演奏タイミング誤差	30 ms 以内	聴感で破綻なし。定量計測は未実施
MOP2	同期信号の受信成功率	95 % 以上	4 子機同時受信を確認。定量計測は未実施
MOP3	テンポ変更への追従時間	1 拍以内	実装済み、演奏中の即時反映を確認
MOP4	音色の再現性	聴衆の過半数が楽器と判断	聴感確認済み。アンケートは進行中
MOP5	歌詞・アニメーション同期遅延	100 ms 以内	LCD 統合が未完のため未計測

検証は単体テスト、結合テスト、システムテストの 3 段階で計画した。単体テストでは各音源と通信機能を個別に確認し、結合テストでは親機・子機間の同期と子機・Processing 間の接続を確認する。システムテストでは全構成を接続して課題曲の通し演奏を行う。

4.2 単体テストの結果

第 5 週から第 8 週にかけて実施した単体テストの結果を表 3 に示す。音源系では、フルートの音色聴感テスト、トランペットの音色合成テストと切り替えノイズテスト、ドラムの単体再生テストがそれぞれ完了した。トランペットの切り替えノイズは、フェードアウト処理の追加によってテスト上で雑音を確認されなくなり、MOP5 に関連する問題点として管理していたものが解消された。通信系では、921600 bps でのシリアル通信テストで 29 音の楽曲データが欠落なく受信・格納されることを確認した。また、赤外線受光モジュールの故障 3 台の交換後に受信テストを再実施し、回路の正常動作を確認している。ピアノ音源は差し替えの際に、楽譜で使う全音高と 2 種類の音長の組について試聴で確認した。

表 3 テストの実施結果

実施日	テスト内容	結果	備考
5/20～5/25	フルート音色聴感テスト	成功	倍音構造・ADSR 調整完了
5/26	トランペット音色合成テスト	成功	倍音比とビブラートの確定
5/26	音切り替えノイズテスト	成功	0.03s フェードアウトで雑音解消
5/26	シリアル通信テスト (921600 bps)	成功	29 音を欠落なく受信
6 月初旬	ドラム音色単体テスト	成功	キック・シンバルの再生確認
6/17	受光モジュール交換後の通信テスト	成功	故障 3 台を交換
6/19	3 子機の輪唱テスト	部分成功	ドラムを除く 3 楽器で動作
6/23	4 子機の輪唱実機テスト	成功	音抜け・休符抜けの再発なし

4.3 結合テストと実機演奏の結果

結合テストは 2 段階で行った。第 7 週末の 3 子機テストでは、ピアノ・フルート・トランペットの 3 楽器での輪唱開始を実機で確認した。この時点ではドラムが未結合であり、部分成功という扱いである。続く第 9 週の 6 月 23 日に、親機 1 台と子機 4 台の全構成による輪唱の実機テストを実施した。ボタンの押下順にピアノ・フルート・トランペット・ドラムが順次参加するカノン演奏を通して行い、聴感上の破綻なく演奏できることを全員で確認した。3.4 節で述べた位置抜けと休符抜けは、このテストで再発していない。参加・退場のトグル操作についても、予約とその取消、途中退場後の再参加が状態表示 LED の点灯と一致して動作した。

テンポ管理については、可変抵抗のつまみ操作で演奏中のテンポが 60～150 BPM の範囲で連続的に変わることを確認した。式 (1) の tick 周期は毎ループ現在のテンポを参照するため、追従の遅れはつまみ読み取りの平滑化に伴う数百 ms 程度であり、1 拍以内という MOP3 の目標を満たす動作である。ただしこの追従時間もログからの定量値としては算出しておらず、シリアルモニタ上の値の変化と聴感での確認にとどまっている。

4.4 定量指標の計測状況

MOP1 と MOP2 の定量計測は、本報告の執筆時点で未実施である。計測の準備として、親機には赤外線送信の直前に同期パルスを出す端子を設け、子機側では割り込みでパルスの立ち上がり時刻を記録し、赤外線フレームの受信復号が完了した時刻との差をマイクロ秒単位でログ出力する機構を実装した。第 9 週の実機テストのログでは受信のたびに遅延値が出力されることを確認しており、第 10 週にこのログから演奏タイミング誤差の統計を取り、あわせて送信回数に対する受信回数の比から受信成功率を算出する計画である。技術性能指標 TPM として監視してきた項目のうち、シリアル通信速度は目標通り動作確認済み、親機と子機の tick 数の整合は 3.4 節の修正で達成済みである。

5 考察

5.1 マスク方式による同期の有効性

本システムの同期方式は、楽譜データを通信せず、参加楽器のマスクと tick 番号だけを毎 tick 送るものである。4 子機の実機テストで輪唱が破綻なく成立したことから、この情報量の絞り込みは有効だったと考える。NEC フォーマット 1 フレームに必要な情報がすべて収まるため、子機の台数を増やしてもフレーム長も送信頻度も変わらない。マスクは 8 ビットあるので、理論上は 8 台までこの構成のまま拡張できる。楽譜を子機に分散させたことは、通信量の削減だけでなく、子機単体でのテストを可能にするという開発上の利点もあった。各楽器の担当者が自分の子機と Processing だけで音源の開発を進められたのは、この構成によるところが大きい。

一方で、この方式は子機が受信回数を数えて自分の位置を進める作りに依存している。赤外線を受信に 1 回失敗すると子機の位置が 1 tick 遅れ、以後もずれたまま演奏が続く。現状の実装では、親機が tick 番号の下位 8 ビットをアドレス部に載せて送っているものの、子機側はまだこの値を位置の補正に使っていない。受信のたびに tick 番号から自分の譜面位置を計算し直すようにすれば、受信失敗が起きても次のフレームで自己修復できる。計測機構とあわせて、今後の改善点の中では優先度が高い。

5.2 tick 数不整合の教訓

3.4 節の位置抜けバグは、親機と子機が「楽譜 1 周分の tick 数」という同じ意味の値を、別々のファイルに別々の定数として持っていたことが根本原因である。修正は定数 1 つの変更で済んだが、症状は数周に一度しか現れないため、原因の特定には子機のログから位置の進み方の周期性を読み取る必要があった。親機 32 に対して子機 33 という 1 tick の差は、単体テストでは決して現れず、実機で長く演奏して初めて聞こえてくる。設計書に定数の対応関係を明記して両側から参照すること、そして結合した状態で数周分を通して聴くテストを早い段階に置くことが、同種の不整合への対策になると考える。

5.3 音長の固定値とテンポ可変の関係

休符抜けの問題は、音長の既定値 0.40 s が tick 間隔 0.25 s より長いという時間設計の矛盾から生じた。休符受信時の即時停止で聴感上は解消したが、これは対症的な処置でもある。本システムはテンポを 60~150 BPM まで連続可変としたので、tick 間隔は式 (1) より 500 ms から 200 ms まで変わる。音長を固定値のままにすると、遅いテンポでは音符間に隙間が生まれ、速いテンポでは余韻の重なりが増える。音長を tick 間隔に比例させて、テンポに追従した長さで発音する方式が本質的な改善になる。子機は直近の受信間隔を計測して倍速区間の遅延に使っており、この値を音長の算出にも流用できるはずである。

5.4 ピアノ音源の方式選択

ピアノ音源で採った事前レンダリング方式は、音色の複雑さと発音遅延の両立という点でうまく働いた。発音時の処理が Sampler の `trigger()` だけになるため、加算合成の計算量が演奏タイミングへ影響しない。一方でこの方式には、音高・音長・音量の組ごとに波形を持つ必要があるという制約がある。今回の楽曲は 6 音高 × 2 音長で済んだが、曲目を増やすならキャッシュの生成時間とメモリが増えていく。また、打鍵の強弱で音色が変わるベロシティ表現は現状持っていない。強弱を数段階の波形として持つか、再生時の音量係数で近似するかは、音質と資源のトレードオフとして今後検討したい。

5.5 システムの限界と改善案

最後に、本システムに残された課題を整理する。評価面では、MOP1 と MOP2 の定量計測が未実施であり、現状の同期品質は聴感による確認にとどまっている。計測機構は実装済みなので、ログの統計処理まで含めて完了させることが最優先である。機能面では、LCD 表示の統合が残っており、歌詞が縦に流れる表示の不具合を修正したうえで、発音との同期遅延を MOP5 の目標 100ms 以内に収める確認が必要である。同期方式については、前述の tick 番号による位置の自己修復を実装すれば、受信失敗への耐性が上がる。さらに、楽譜が子機のプログラムに固定配列として埋め込まれている点も、曲目の追加を考えると改善の余地がある。楽譜データを外部から書き換えられる形式にすれば、同じハードウェア構成のまま演奏曲を増やせる。

6 おわりに

本プロジェクトでは、複数の機器をリアルタイムに同期させる仕組みを合奏という題材で実現することを目的として、赤外線によるマスク同期を用いた 5 台構成の Arduino 輪唱演奏システムを制作した。親機が参加楽器のマスクと tick 番号を毎 tick 一斉送信し、楽譜を保持する各子機が自分の担当ビットを参照して演奏する分散構成により、楽曲データの通信なしで「かえるのうた」の輪唱を成立させた。演奏者はボタンで各楽器を任意のタイミングで参加・退場させることができ、可変抵抗によるテンポの連続調整にも対応した。

報告者は、倍音の加算合成と事前レンダリングによるピアノ音源、ドラム子機のプログラム、可変抵抗によるテンポ管理を実装し、親機と子機の tick 数不整合による周期的な音抜けと、音の余韻による休符抜けの 2 つの不具合を修正した。第 9 週の実機テストでは、親機 1 台と子機 4 台による押下順カノン演奏が音抜けや休符抜けなく成立することを確認し、序論で述べた数十 ms 以内の状態共有という要件を、聴感上の破綻がないという水準で満たすことができた。

一方、演奏タイミング誤差 30ms 以内と受信成功率 95% 以上という定量目標については、計測機構の実装までにとどまり、計測値による裏付けが残された課題である。今後は、実装済みの遅延ログから同期精度を定量的に評価するとともに、tick 番号を用いた譜面位置の自己修復、テンポに追従する音長制御、および LCD 表示の統合を進め、実演可能な完成度へ引き上げていきたい。

参考文献

- [1] Rasch, R. A., “Synchronization in performed ensemble music,” *Acustica*, Vol.43, No.2, pp.121–131, 1979.
- [2] 一般社団法人音楽電子事業協会 (AMEI), “MIDI 規格,” <https://amei.or.jp/midistandardcommittee/>, 参照 2026 年 6 月.
- [3] SB-Projects, “IR Remote Control Theory: NEC Protocol,” <https://www.sbprojects.net/knowledge/ir/nec.php>, 参照 2026 年 5 月.
- [4] Arduino-IRremote Community, “IRremote Arduino Library,” <https://github.com/Arduino-IRremote/Arduino-IRremote>, 参照 2026 年 5 月.
- [5] Di Fede, D., “Minim: An audio library for Processing,” <http://code.compartmental.net/minim/>, 参照 2026 年 5 月.
- [6] Arduino, “Arduino Documentation,” <https://docs.arduino.cc/>, 参照 2026 年 5 月.
- [7] OptoSupply, “OSRB38C9AA Infrared Receiver Module Datasheet,” 参照 2026 年 6 月.
- [8] Fletcher, H., Blackham, E. D. and Stratton, R., “Quality of Piano Tones,” *The Journal of the Acoustical Society of America*, Vol.34, No.6, pp.749–761, 1962.
- [9] 西口磯春, “ピアノの音響とその物理モデル,” *RIST ニュース*, No.56, pp.3–15, 2014.
- [10] Arrighi, R., Alais, D. and Burr, D., “Perceptual synchrony of audiovisual streams for natural and artificial motion sequences,” *Journal of Vision*, Vol.6, No.3, pp.260–268, 2006.

A 親機 Arduino のスケッチ

親機のスケッチを以下に示す。報告者の担当箇所は、可変抵抗によるテンポ管理を行う `handleTempoPot()` と、子機の状態表示 LED の制御である。

```
1  #define DECODE_NEC
2  #define IR_SEND_PIN 3
3  #include <IRremote.hpp>
4
5  const int NUM_CHILDREN = 4;
6
7  const int PIN_BTN_ORDER[NUM_CHILDREN] = {4, 5, 6, 7}; // 子機ごとの「参加/退場」トグルボタン
8  // 各子機ボタン横のLED (D4→A0, D5→A1, D6→A2, D7→A3)。
9  // ボタン押下毎に必ずトグル (参加予約/取消/退場予約/取消 のいずれでも反転)。
10 const int PIN_LED_CHILD[NUM_CHILDREN] = {A0, A1, A2, A3};
11 const int PIN_BTN_START = 8;
12 const int PIN_POT_TEMPO = A5; // BPM可変抵抗器 (uxcell 10kΩ シングルターン)。回転で 60~150 BPM
13 const int PIN_BTN_RESET = 10;
14 const int LED_INDICATOR = 13;
15 const int PIN_SYNC_OUT = 9; // 子機D3への同期パルス出力 (遅延計測用)
16
17 const int QUANTIZE_TICKS = 8; // 参加予約のアライメント(0,8,16,...)
18 // 子機の TOTAL_TICKS (=楽譜1周分のIR tick数) と一致させる必要がある。
19 // 退場予約はこの倍数(子機の譜面ループ末尾)にアライメントする。
20 const int SCORE_TICKS = 32;
21 const int TEMPO_MIN_BPM = 60;
22 const int TEMPO_MAX_BPM = 150;
23 const unsigned long POT_READ_INTERVAL_MS = 50;
24 // NOTE: address フィールドは tickCount下位8bit用に転用するため、固定アドレスとしては未使用
25
26 bool isActive[NUM_CHILDREN] = {false, false, false, false}; // 参加済みフラグ
27 long joinTick[NUM_CHILDREN] = {-1, -1, -1, -1}; // 参加予定tick (未予約は-1)
28 long leaveTick[NUM_CHILDREN] = {-1, -1, -1, -1}; // 退場予定tick (未予約は-1)
29 long joinedAtTick[NUM_CHILDREN] = {-1, -1, -1, -1}; // 実際に参加発火したtick(退場アライメント計算用)
30
31 bool isPlaying = false;
32 int tempoBpm = 120;
33 long tickCount = 0;
```

```

34 unsigned long lastTickMs = 0;
35
36 int prevBtnOrder[NUM_CHILDREN] = {HIGH, HIGH, HIGH, HIGH};
37 int prevBtnStart = HIGH;
38 int prevBtnReset = HIGH;
39
40 // ポテンショメータ読み取りの平滑化と更新間引き用
41 unsigned long lastPotReadMs = 0;
42 int smoothedPotRaw = -1;
43 int lastReportedBpm = -1;
44
45 void setup() {
46     Serial.begin(115200);
47     IrSender.begin(IR_SEND_PIN);
48
49     for (int i = 0; i < NUM_CHILDREN; i++) {
50         pinMode(PIN_BTN_ORDER[i], INPUT_PULLUP);
51         pinMode(PIN_LED_CHILD[i], OUTPUT);
52         digitalWrite(PIN_LED_CHILD[i], LOW);
53     }
54     pinMode(PIN_BTN_START, INPUT_PULLUP);
55     pinMode(PIN_BTN_RESET, INPUT_PULLUP);
56     // PIN_POT_TEMPO はアナログ入力なので pinMode 不要 (pull-up は付けない)
57     pinMode(LED_INDICATOR, OUTPUT);
58     pinMode(PIN_SYNC_OUT, OUTPUT);
59     digitalWrite(PIN_SYNC_OUT, LOW);
60
61     printHelp();
62 }
63
64 void loop() {
65     handleSerialCommand();
66     handleOrderButtons();
67     handleResetButton();
68     handleTempoPot();
69     handleStartButton();
70
71     if (isPlaying) {
72         unsigned long now = millis();
73         unsigned long interval = 60000UL / (unsigned long)tempoBpm / 2UL;
74         if (now - lastTickMs >= interval) {
75             lastTickMs = now;
76             tick();
77             tickCount++;
78         }
79     }
80
81     // 子機LEDを内部状態に同期。isActive XOR (joinTick≠-1 || leaveTick≠-1) で

```



```

82 // 「参加意思あり=点灯」となり、ボタン押下毎に確実にトグルする。
83 // 4状態の遷移は tick() で join/leave 反映後も継続する:
84 // 未参加+予約なし(OFF) → 未参加+joinTick(ON) → tickでjoin → 参加+予約なし(ON)
85 // → 参加+leaveTick(OFF) → tickでleave → 未参加+予約なし(OFF)
86 for (int i = 0; i < NUM_CHILDREN; i++) {
87     bool on = isActive[i] != ((joinTick[i] != -1) || (leaveTick[i] != -1));
88     digitalWrite(PIN_LED_CHILD[i], on ? HIGH : LOW);
89 }
90 }
91
92 // 参加ボタン（子機ごと）のトグル処理（ボタン・シリアル共有）。
93 // 未参加 → 次の区切りから参加予約
94 // 参加予約中 → その予約を取消（参加しない）
95 // 参加中 → 次の区切りから退場予約
96 // 退場予約中 → その予約を取消（残留）
97 void toggleChild(int i) {
98     if (i < 0 || i >= NUM_CHILDREN) return;
99     if (!isPlaying) {
100         Serial.println("[!]_再生中のみ操作できます");
101         return;
102     }
103
104     long joinBoundary = ((tickCount / QUANTIZE_TICKS) + 1) * QUANTIZE_TICKS;
105
106     if (isActive[i]) {
107         if (leaveTick[i] != -1) {
108             leaveTick[i] = -1;
109             Serial.print("child_");
110             Serial.print(i);
111             Serial.println("_leave_cancelled_(stay)");
112         } else {
113             // 子機が現在のループを最後まで弾き切ってから抜けるよう、
114             // joinedAtTick から数えて次の SCORE_TICKS 境界に揃える。
115             long ticksSinceJoin = tickCount - joinedAtTick[i];
116             long loopsCompleted = ticksSinceJoin / SCORE_TICKS;
117             leaveTick[i] = joinedAtTick[i] + (loopsCompleted + 1) * SCORE_TICKS;
118             Serial.print("child_");
119             Serial.print(i);
120             Serial.print("_will_leave_at_tick_");
121             Serial.print(leaveTick[i]);
122             Serial.print("_end_of_loop_");
123             Serial.print(loopsCompleted + 1);
124             Serial.println("");
125         }
126     } else {
127         if (joinTick[i] != -1) {
128             joinTick[i] = -1;
129             Serial.print("child_");

```

```

130     Serial.print(i);
131     Serial.println("_join_cancelled");
132 } else {
133     joinTick[i] = joinBoundary;
134     Serial.print("child_");
135     Serial.print(i);
136     Serial.print("_will_join_at_tick_");
137     Serial.println(joinTick[i]);
138 }
139 }
140 }
141
142 // 参加ボタン：押すごとに 参加予約⇔退場予約 をトグルする（リアルタイム、次の区切りに
    反映）
143 void handleOrderButtons() {
144     for (int i = 0; i < NUM_CHILDREN; i++) {
145         int s = digitalRead(PIN_BTN_ORDER[i]);
146         if (prevBtnOrder[i] == HIGH && s == LOW) {
147             toggleChild(i);
148             delay(30);
149         }
150         prevBtnOrder[i] = s;
151     }
152 }
153
154 void doReset() {
155     isPlaying = false;
156     tickCount = 0;
157     for (int i = 0; i < NUM_CHILDREN; i++) {
158         isActive[i] = false;
159         joinTick[i] = -1;
160         leaveTick[i] = -1;
161         joinedAtTick[i] = -1;
162     }
163     digitalWrite(LED_INDICATOR, LOW);
164     Serial.println("reset");
165 }
166
167 void handleResetButton() {
168     int s = digitalRead(PIN_BTN_RESET);
169     if (prevBtnReset == HIGH && s == LOW) {
170         doReset();
171         delay(30);
172     }
173     prevBtnReset = s;
174 }
175
176 // 可変抵抗（A5）の値を 60~150 BPM に線形マップしてリアルタイム反映する。

```

```

177 // 50ms間隔で読み取り、指数移動平均で軽くノイズを抑える。
178 // 既存の tick 周期計算は毎ループ tempoBpm を参照しているため、即座に反映される。
179 void handleTempoPot() {
180     unsigned long now = millis();
181     if (now - lastPotReadMs < POT_READ_INTERVAL_MS) return;
182     lastPotReadMs = now;
183
184     int raw = analogRead(PIN_POT_TEMPO);
185     if (smoothedPotRaw < 0) smoothedPotRaw = raw;
186     else smoothedPotRaw = (smoothedPotRaw * 3 + raw) / 4;
187
188     int newBpm = map(smoothedPotRaw, 0, 1023, TEMPO_MIN_BPM, TEMPO_MAX_BPM);
189     newBpm = constrain(newBpm, TEMPO_MIN_BPM, TEMPO_MAX_BPM);
190     tempoBpm = newBpm;
191
192     // 微小変動でシリアルが埋まらないよう、2BPM以上変わったときだけログ
193     if (lastReportedBpm < 0 || abs(newBpm - lastReportedBpm) >= 2) {
194         Serial.print("tempo=");
195         Serial.println(tempoBpm);
196         lastReportedBpm = newBpm;
197     }
198 }
199
200 // 再生開始：誰も参加していない状態から開始する（参加はボタンで都度行う）
201 void doStart() {
202     if (isPlaying) {
203         Serial.println("[!]_already_playing");
204         return;
205     }
206     isPlaying = true;
207     tickCount = 0;
208     lastTickMs = millis() - (60000UL / (unsigned long)tempoBpm / 2UL);
209     for (int i = 0; i < NUM_CHILDREN; i++) {
210         isActive[i] = false;
211         joinTick[i] = -1;
212         leaveTick[i] = -1;
213         joinedAtTick[i] = -1;
214     }
215     Serial.println("start");
216 }
217
218 void handleStartButton() {
219     int s = digitalRead(PIN_BTN_START);
220     if (prevBtnStart == HIGH && s == LOW) {
221         doStart();
222         delay(30);
223     }
224     prevBtnStart = s;

```

```

225 }
226
227 void handleSerialCommand() {
228     while (Serial.available()) {
229         char c = (char)Serial.read();
230         if (c == '\r' || c == '\n' || c == '_') continue;
231
232         if (c >= '0' && c <= ('0' + NUM_CHILDREN - 1)) {
233             toggleChild(c - '0');
234         } else if (c == 's' || c == 'S') {
235             doStart();
236         } else if (c == 'r' || c == 'R') {
237             doReset();
238         } else if (c == '?') {
239             printHelp();
240         }
241     }
242 }
243
244 void printHelp() {
245     Serial.println("===_hack_oya_kai_help_===");
246     Serial.println("0..3_: 子機の参加/退場トグル_(D4..D7_ボタンと同じ動作)");
247     Serial.println("_____未参加→参加予約_/_参加中→退場予約_/_予約中に再度押すと取消");
248     Serial.println("s_____:_start");
249     Serial.println("r_____:_reset_(即時の強制停止)");
250     Serial.println("?_____:_help");
251     Serial.println("(BPM_は_A5_のポテンシヨメータで_60-150_を連続調整)");
252     Serial.print("Current_BPM=");
253     Serial.println(tempoBpm);
254 }
255
256 void tick() {
257     digitalWrite(LED_INDICATOR, (tickCount % 2) ? HIGH : LOW);
258
259     uint8_t mask = 0;
260     for (int i = 0; i < NUM_CHILDREN; i++) {
261         // 参加予約していたtickに到達したら有効化
262         if (!isActive[i] && joinTick[i] != -1 && tickCount >= joinTick[i]) {
263             isActive[i] = true;
264             joinTick[i] = -1;
265             joinedAtTick[i] = tickCount; // 退場アライメント計算の起点
266             Serial.print("child_");
267             Serial.print(i);
268             Serial.println("_joined");
269         }
270         // 退場予約していたtickに到達したら無効化 (このtickからは送らない)
271         if (isActive[i] && leaveTick[i] != -1 && tickCount >= leaveTick[i]) {

```

```

272     isActive[i] = false;
273     leaveTick[i] = -1;
274     joinedAtTick[i] = -1;
275     Serial.print("child_");
276     Serial.print(i);
277     Serial.println("_left");
278 }
279 if (isActive[i]) {
280     mask |= (uint8_t)(1 << i);
281 }
282 }
283
284 if (mask != 0) {
285     // 子機D3への同期パルス。IR送信直前に立てて、子機ISRのmicros()と
286     // 本ループのsendNEC完了までの遅延を子機側で計測できるようにする。
287     digitalWrite(PIN_SYNC_OUT, HIGH);
288     delayMicroseconds(50);
289     digitalWrite(PIN_SYNC_OUT, LOW);
290
291     // address に tickCount の下位8bit を乗せて送る。
292     // 子機側はこれを「正解の位置」として受け取り、localPos を補正する
293     // (IR受信失敗による同期ずれを毎tick自己修復できる)。
294     uint8_t tickLow = (uint8_t)(tickCount & 0xFF);
295     IrSender.sendNEC(tickLow, mask, 0);
296 }
297 }

```

B 子機 Arduino のスケッチ

ピアノ・フルート・トランペット・ドラムの4子機で共通のスケッチを以下に示す。書き込み時に `childId` を機体ごとに変更する。

```

1  #define DECODE_NEC
2  #include <IRremote.hpp>
3
4  // hack_oya_kai.ino 用プロトコル対応版:
5  //   IrSender.sendNEC(IR_ADDRESS=0x00, mask, 0) を受信。
6  //   mask の bit(childId) が立っているtickだけ「自分の出番」として
7  //   localPos をカウントアップし、その位置の音をホストへ送信する。
8  //   親機側にループの概念がないが、メロディ機 (childId != DRUM_CHILD_ID)
9  //   は楽譜の最後まで行ったら localPos を 0 に戻し、自力でループする。
10 //   (ドラム機は元々楽譜を持たず鳴らし続けるだけなので、この対象外)
11
12 // 書き込む機体ごとに値を変えて Upload する: 0,1,2 = メロディ機, 3 = ドラム機。
13 int childId = 0;
14 const int DRUM_CHILD_ID = 3;
15 const int PIN_IR_RECV = 2;

```

```

16 const int PIN_SYNC_IN = 3; // 親機D9からの同期パルス入力（遅延計測用）
17 const int LED_INDICATOR = 13;
18
19 const int SCORE_LENGTH = 40;
20 const String myScore[SCORE_LENGTH] = {
21     "C4", "D4", "E4", "F4", "E4", "D4", "C4", "R",
22     "E4", "F4", "G4", "A4", "G4", "F4", "E4", "R",
23     "C4", "R", "C4", "R", "C4", "R", "C4", "R",
24     "C4", "C4", "D4", "D4", "E4", "E4", "F4", "F4",
25     "E4", "R", "D4", "R", "C4", "R", "R", "R"
26 };
27
28 // index 24 以降は1IR=2音(16分音符相当)で発音する。
29 const int DOUBLE_START = 24;
30
31 // 1周(ループ)に必要なtick数。
32 // 通常区間: 0 .. DOUBLE_START-1 (1tick=1音)
33 // 倍速区間: DOUBLE_START .. (1tick=2音) なので tick数は半分になる
34 // 例) DOUBLE_START=24, SCORE_LENGTH=40 → 24 + (40-24)/2 = 32
35 const int TOTAL_TICKS = DOUBLE_START + (SCORE_LENGTH - DOUBLE_START) / 2;
36
37 // Processing への通信フォーマット: "ピッチ名,duration秒,amplitude,localPos\n"
38 const float NOTE_DURATION_DEFAULT = 0.40f;
39 const float NOTE_DURATION_HALF = 0.20f;
40 const float NOTE_AMPLITUDE = 0.60f;
41
42 // ドラム機が drum.pde に送るキック合図 (drum.pde は "C2" 単独行を期待)。
43 const String DRUM_NOTE = "C2";
44
45 // 自分の出番が来た回数 (= 親機のtickCountに相当)。受信したtickごとに進む。
46 long localPos = -1; // -1 = まだ一度も自分の出番が来ていない
47
48 unsigned long lastReceiveMs = 0;
49 unsigned long lastIntervalMs = 250; // 双子音化区間の半tick遅延に使う
50
51 // 遅延計測: 親機の同期パルス立ち上がり時刻をISRで記録
52 volatile unsigned long syncRiseUs = 0;
53
54 void onSyncRise() {
55     syncRiseUs = micros();
56 }
57
58 void sendNoteToHost(const String& pitch, float duration, float amplitude, long pos) {
59     Serial.print(pitch);
60     Serial.print(',');
61     Serial.print(duration, 3);
62     Serial.print(',');
63     Serial.print(amplitude, 3);

```

```

64     Serial.print(',');
65     Serial.println(pos);
66 }
67
68 void printReady() {
69     Serial.print("hack-ko_ready_CHILD_ID=");
70     Serial.print(childId);
71     Serial.println(childId == DRUM_CHILD_ID ? "_(DRUM)" : "_(MELODY)");
72 }
73
74 void setup() {
75     Serial.begin(115200);
76
77     // UNO R4 等の native USB CDC は Serial.begin() 直後はホスト未接続のことがある。
78     // 最大3秒だけCDC接続完了を待つ (タイムアウトしてもそのまま進む)。
79     unsigned long boot_t = millis();
80     while (!Serial && (millis() - boot_t) < 3000) { ; }
81
82     IrReceiver.begin(PIN_IR_RECV);
83     pinMode(PIN_SYNC_IN, INPUT);
84     attachInterrupt(digitalPinToInterrupt(PIN_SYNC_IN), onSyncRise, RISING);
85     pinMode(LED_INDICATOR, OUTPUT);
86
87     delay(100);
88     printReady();
89 }
90
91 void loop() {
92     if (!IrReceiver.decode()) return;
93
94     // decode直後にデータを退避し、即座にresumeする。
95     // delay() 中もIR受信を継続させ、tick取りこぼしを防ぐ。
96     uint8_t protocol = IrReceiver.decodedIRData.protocol;
97     uint8_t mask = (protocol == NEC) ? IrReceiver.decodedIRData.command : 0;
98     IrReceiver.resume();
99
100    if (protocol == NEC) {
101        if (mask & (1 << childId)) {
102            unsigned long recvUs = micros();
103            unsigned long riseUs = syncRiseUs; // ISRで記録された値をコピー
104            Serial.print("[RECV]_millis=");
105            Serial.print(millis());
106            Serial.print("_child=");
107            Serial.print(childId);
108            Serial.print("_localPos=");
109            Serial.print(localPos + 1);
110            if (riseUs != 0) {
111                Serial.print("_latency=");

```

```

112     Serial.print(recvUs - riseUs);
113     Serial.print("us");
114 }
115 Serial.println();
116 localPos++;
117
118 if (childId == DRUM_CHILD_ID) {
119     Serial.println(DRUM_NOTE);
120     digitalWrite(LED_INDICATOR, HIGH);
121     delay(15);
122     digitalWrite(LED_INDICATOR, LOW);
123 } else {
124     if (localPos >= TOTAL_TICKS) {
125         localPos = 0;
126     }
127
128     unsigned long now = millis();
129     if (lastReceiveMs != 0) {
130         unsigned long interval = now - lastReceiveMs;
131         // 演奏停止→再開時の大きなギャップでdelayが膨張するのを防ぐ
132         if (interval >= 50 && interval <= 2000 && interval <= lastIntervalMs * 3) {
133             lastIntervalMs = interval;
134         }
135     }
136     lastReceiveMs = now;
137
138     if (localPos >= 0 && localPos < DOUBLE_START) {
139         sendNoteToHost(myScore[localPos], NOTE_DURATION_DEFAULT, NOTE_AMPLITUDE,
140             localPos);
141         digitalWrite(LED_INDICATOR, (localPos % 2) ? HIGH : LOW);
142     } else if (localPos >= DOUBLE_START) {
143         int scoreIdx = DOUBLE_START + 2 * (int)(localPos - DOUBLE_START);
144         if (scoreIdx < SCORE_LENGTH) {
145             sendNoteToHost(myScore[scoreIdx], NOTE_DURATION_HALF, NOTE_AMPLITUDE,
146                 localPos);
147             digitalWrite(LED_INDICATOR, (localPos % 2) ? HIGH : LOW);
148         }
149         if (scoreIdx + 1 < SCORE_LENGTH) {
150             unsigned long halfDelay = min(lastIntervalMs / 2, 200UL);
151             delay(halfDelay);
152             sendNoteToHost(myScore[scoreIdx + 1], NOTE_DURATION_HALF, NOTE_AMPLITUDE,
153                 localPos);
154         }
155     }
156 }

```


C ピアノ音再生用 Processing プログラム

ピアノ音の合成と再生を行う Processing のプログラムを以下に示す。

```
1  import processing.serial.*;
2  Serial port;
3  import ddf.minim.*;
4  import ddf.minim.ugens.*;
5  import java.util.HashMap;
6  Minim minim;
7  AudioOutput out;
8  String currentNote = "";
9  int currentPos = -1; // 親機が割り当ててきた localPos (デバッグ表示用)
10
11 // ---- グランドピアノ音源 (音源部のみ差し替え。シリアル受信の仕組みは既存のまま)
12 // ----
13 // 方式: 既知ピッチは起動時にプリレンダしてSamplerにキャッシュ、発音はtrigger()のみ。
14 HashMap<String, Sampler> pianoCache = new HashMap<String, Sampler>();
15 Sampler lastSampler = null; // 直前に鳴らした音。R受信時にstop()で即打ち切る (旧
16   noteOff()と同じ役割)
17
18 final float SAMPLE_RATE = 44100f;
19 final float RELEASE_TAIL = 0.8f; // ノート長に加えてレンダリングする余韻(秒)
20
21 void setup() {
22   size(512, 200);
23   minim = new Minim(this);
24   out = minim.getLineOut();
25
26   // 楽譜(hack-koP2.ino)で使うピッチ×2種の長さを起動時にプリレンダリング。
27   // 未知のピッチ/長さ/音量が来ても playNote 側の安全網で都度生成される。
28   // ポートを開く前に済ませ、プリレンダ中のシリアル取りこぼしを避ける。
29   String[] knownPitches = { "C4", "D4", "E4", "F4", "G4", "A4" };
30   float[] knownDurs = { 0.40f, 0.20f };
31   long t0 = millis();
32   println("ピアノ音源をプリレンダリング中...");
33   for (String pitch : knownPitches) {
34     for (float dur : knownDurs) {
35       String key = cacheKey(pitch, dur, 0.60f);
36       pianoCache.put(key, buildSampler(pitch, dur, 0.60f));
37       println("_" + key + "_OK");
38     }
39   }
40   println("プリレンダ完了:_" + pianoCache.size() + "音_/_" + (millis() - t0) + "ms");
41 }
```

```

39 // portは各自変えて.
40 port = new Serial(this, "/dev/cu.usbserial-A5069RR4", 115200);
41 port.clear();
42 port.bufferUntil('\n');
43 }
44
45 void draw() {
46   background(0);
47   stroke(255);
48   for (int i = 0; i < out.bufferSize() - 1; i++) {
49     line(i, 50 + out.left.get(i)*50, i+1, 50 + out.left.get(i+1)*50);
50   }
51   fill(255);
52   text("note:_" + currentNote, 10, 170);
53   text("pos:_" + currentPos, 10, 190);
54 }
55
56 void serialEvent(Serial p) {
57   String inString = p.readStringUntil('\n');
58   if (inString == null) return;
59   inString = trim(inString);
60   if (inString.length() == 0) return;
61   currentNote = inString;
62
63   // ino側のフォーマット: "ピッチ名,duration秒,amplitude,localPos"
64   // localPos は親機が決めた楽譜index。発音には使わずデバッグ表示専用。
65   // 旧3列フォーマット(pitch,duration,amplitude)も互換のため受け付ける。
66   // それ以外(起動メッセージなど)はパース不能としてスキップ。
67   String[] parts = split(inString, ',');
68   if (parts.length < 3) {
69     println("non-note:_" + inString);
70     return;
71   }
72   String pitch = parts[0];
73   float duration = float(parts[1]);
74   float amplitude = float(parts[2]);
75   if (parts.length >= 4) {
76     currentPos = int(parts[3]);
77   }
78   // R は休符: 直前の音の余韻が次のtickまで残らないよう、即stop()で打ち切る。
79   // (NOTE_DURATION_DEFAULT=0.40s は tick間隔(120BPMで0.25s)より長いので、
80   // R が来た時点で前音がまだ鳴っているのを止めないと休符に聞こえない。)
81   if (pitch.equals("R")) {
82     if (lastSampler != null) {
83       lastSampler.stop();
84       lastSampler = null;
85     }
86     return;

```

```

87     }
88     playNote(pitch, duration, amplitude);
89 }
90
91 void playNote(String pitch, float duration, float amplitude) {
92     String key = cacheKey(pitch, duration, amplitude);
93     Sampler smp = pianoCache.get(key);
94     if (smp == null) { // 安全網: 未キャッシュの組み合わせは都度生成してキャッシュ
95         smp = buildSampler(pitch, duration, amplitude);
96         pianoCache.put(key, smp);
97     }
98     smp.trigger();
99     lastSampler = smp;
100 }
101
102 String cacheKey(String pitch, float duration, float amplitude) {
103     return pitch + "_" + nf(duration, 0, 2) + "_" + nf(amplitude, 0, 2);
104 }
105
106 // ---- グランドピアノ音源 ----
107 Sampler buildSampler(String pitch, float dur, float amp) {
108     float f0 = Frequency.ofPitch(pitch).asHz();
109     float[] data = renderPianoNote(f0, dur, amp);
110     MultiChannelBuffer mcb = new MultiChannelBuffer(data.length, 1);
111     mcb.setChannel(0, data);
112     Sampler smp = new Sampler(mcb, SAMPLE_RATE, 6); // 最大6声 (余韻の重なり用)
113     smp.patch(out);
114     return smp;
115 }
116
117 float[] renderPianoNote(float f0, float dur, float amp) {
118     int n = (int)((dur + RELEASE_TAIL) * SAMPLE_RATE);
119     float[] buf = new float[n];
120
121     // 複弦構成 (実ピアノ: 高音ほど弦が多い)。デチューンはcent単位
122     float[] cents;
123     if (f0 >= 350f) cents = new float[] { 0f, 1.2f, -0.8f };
124     else if (f0 >= 150f) cents = new float[] { 0f, 1.0f };
125     else cents = new float[] { 0f };
126
127     float p = 0.9f + 0.5f * constrain((f0 - 110f) / 770f, 0f, 1f); // 倍音ロール
128     float B = 0.0002f * pow(f0 / 261.6f, 0.8f); // インハーモ
129     float T60 = constrain(6.0f * pow(261.6f / f0, 0.7f), 0.8f, 8.0f); // 残響長(音
130     float tauBase = T60 / 6.9f;
131     float stringAmp = amp / cents.length;

```

```

132
133     for (int s = 0; s < cents.length; s++) {
134         float fs = f0 * pow(2f, cents[s] / 1200f);
135         for (int h = 1; h <= 10; h++) {
136             float fh = h * fs * sqrt(1f + B * h * h);    // インハーモニシティ補正周波数
137             if (fh > SAMPLE_RATE * 0.45f) break;
138             float ah = stringAmp / pow(h, p);
139             float tauH = tauBase / (1f + 0.6f * (h - 1)); // 高次倍音ほど速く減衰
140             float w = TWO_PI * fh / SAMPLE_RATE;
141             for (int i = 0; i < n; i++) {
142                 float t = i / SAMPLE_RATE;
143                 float env = 0.3f * exp(-t / 0.12f) + 0.7f * exp(-t / tauH); // 2段階減衰
144                 buf[i] += ah * env * sin(w * i);
145             }
146         }
147     }
148
149     // ハンマーノイズ: 最初の15ms、1次LPF(fc=f0*8上限8k)、指数減衰
150     float fc = min(f0 * 8f, 8000f);
151     float a = 1f - exp(-TWO_PI * fc / SAMPLE_RATE);
152     float lp = 0f;
153     int nNoise = (int)(0.015f * SAMPLE_RATE);
154     randomSeed(12345); // 打鍵音の再現性
155     for (int i = 0; i < nNoise && i < n; i++) {
156         float t = i / SAMPLE_RATE;
157         float x = random(-1f, 1f);
158         lp += a * (x - lp);
159         buf[i] += amp * 0.15f * lp * exp(-t / 0.008f);
160     }
161
162     // アタック(3ms)とダンパーリリース(dur経過後  $\tau=0.15s$ )
163     for (int i = 0; i < n; i++) {
164         float t = i / SAMPLE_RATE;
165         buf[i] *= (1f - exp(-t / 0.003f));
166         if (t > dur) buf[i] *= exp(-(t - dur) / 0.15f);
167     }
168
169     // クリップ防止
170     float peak = 0f;
171     for (int i = 0; i < n; i++) peak = max(peak, abs(buf[i]));
172     if (peak > 0.98f) {
173         float g = 0.98f / peak;
174         for (int i = 0; i < n; i++) buf[i] *= g;
175     }
176     return buf;
177 }

```